

0 x 4 1

Il calcolatore e i linguaggi di programmazione

Numeri reali, caratteri, immagini, istruzioni macchina e compilazione

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027



SAPIENZA
UNIVERSITÀ DI ROMA

Doppia precisione: il double

Stessa idea del float, ma con 64 bit

double (64 bit)



float (32 bit)



stessa struttura: $(-1)^s \times m \times 2^e$

	float	double
esponente	8 bit, $E = e + 127$	11 bit, $E = e + 1023$
mantissa	23 bit	52 bit
cifre decimali significative	circa 7	circa 16

Quali numeri possiamo rappresentare?

Intervalli dei tipi numerici in C (per i reali, in valore assoluto)

tipo C	bit	minimo	massimo
unsigned int	32	0	4 294 967 295
int	32	-2 147 483 648	2 147 483 647
long long	64	-9.2×10^{18}	9.2×10^{18}
float	32	1.4×10^{-45}	3.4×10^{38}
double	64	4.9×10^{-324}	1.8×10^{308}

■ interi ■ reali (virgola mobile)

Il più piccolo float positivo: $2^{-23} \times 2^{-126} = 2^{-149}$

Valori speciali

Gli esponenti riservati $E = 0$ e $E = 255$ (float) codificano zero, infinito e NaN

zero

esponente

00000000

mantissa

0...0

tutti i bit a 0

es. 0

+ infinito

esponente

11111111

mantissa

0...0

$s = 0$

es. 1/0

– infinito

esponente

11111111

mantissa

0...0

$s = 1$

es. -1/0

NaN

esponente

11111111

mantissa

$\neq 0$

Not a Number

es. 0/0, $\sqrt{-1}$



Output di approx.c

```
1/0      = inf
-1/0     = -inf
sqrt(-1) = nan
```

Costanti nella libreria matematica

`M_PI` = π

`M_PI_2` = $\pi/2$

Un problema di approssimazione

Sottrazione di due numeri vicini, con 3 cifre decimali in base 10

Vogliamo calcolare $a - b$ con $a = 1000$ e $b = 999.8$

forma normale, 3 cifre dopo la virgola

$$\begin{aligned} a &= 1.000 \times 10^3 \\ b &= 0.9998 \times 10^3 \rightarrow 0.999 \times 10^3 \end{aligned}$$

L'ultima cifra di b non ci sta: viene scartata (arrotondata).

risultato

$$a - b = (1.000 - 0.999) \times 10^3 = 0.001 \times 10^3 = 1$$

Il risultato esatto è 0.2: abbiamo sbagliato di un fattore 5.

Più cifre, errore più piccolo

Il problema è intrinseco al calcolo con cifre (o bit) in numero finito

$$\begin{aligned}a &= 1.0000 \times 10^3 \\ b &= 0.9998 \times 10^3 \text{ (esatto)}\end{aligned}$$

$$a - b = 0.0002 \times 10^3 = 0.2$$

Con 4 cifre il risultato è giusto.

Ma ci sono sempre numeri per cui l'approssimazione si fa sentire:

0.1 in base 2 è periodico, come 1/3 in base 10.

Lo stesso, in C con double

```
a - b = 0.200000000000004547
0.1+0.2 = 0.30000000000000004
0.1+0.2 == 0.3 ? 0 (falso!)
```

Mai confrontare due reali con `==`: si guarda se la differenza è piccola.

Rappresentare i caratteri: codice ASCII

Una tabella di corrispondenza tra caratteri e numeri interi (anni '60)

carattere	decimale	esadecimale	8 bit
A	65	0x41	01000001
B	66	0x42	01000010
a	97	0x61	01100001
z	122	0x7A	01111010
0	48	0x30	00110000
␣	32	0x20	00100000

␣ = spazio. La tabella completa ha 128 caratteri, 0 ... 127.

$$2^8 = 256$$

combinazioni in 1 byte

0 ... 127 comuni

128 ... 255 estensioni

```
In C
char c = 'A';
printf("%d", c);
→ 65
```

La tabella ASCII completa

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	NUL	32	20	SP	64	40	@	96	60	`
1	1	SOH	33	21	!	65	41	A	97	61	a
2	2	STX	34	22	"	66	42	B	98	62	b
3	3	ETX	35	23	#	67	43	C	99	63	c
4	4	EOT	36	24	\$	68	44	D	100	64	d
5	5	ENQ	37	25	%	69	45	E	101	65	e
6	6	ACK	38	26	&	70	46	F	102	66	f
7	7	BEL	39	27	'	71	47	G	103	67	g
8	8	BS	40	28	(	72	48	H	104	68	h
9	9	HT	41	29	)	73	49	I	105	69	i
10	A	LF	42	2A	*	74	4A	J	106	6A	j
11	B	VT	43	2B	+	75	4B	K	107	6B	k
12	C	FF	44	2C	,	76	4C	L	108	6C	l
13	D	CR	45	2D	-	77	4D	M	109	6D	m
14	E	SO	46	2E	.	78	4E	N	110	6E	n
15	F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

0–31 e 127 sono caratteri di controllo: **LF** = a capo (\n), **HT** = tabulazione (\t), **SP** = spazio.

Unicode: tutti gli alfabeti del mondo

Con 256 caratteri non basta: accenti, greco, cirillico, cinese, simboli ...

$$17 \times 2^{16}$$

= 1 114 112 caratteri possibili
circa 150 000 già assegnati

Compatibile con ASCII

i primi 256 caratteri di Unicode
coincidono con il codice ASCII

A	U+0041	65
---	--------	----

è	U+00E8	232
---	--------	-----

π	U+03C0	960
---	--------	-----

€	U+20AC	8364
---	--------	------

∞	U+221E	8734
---	--------	------

Immagini: pixel e colori RGB

Un'immagine è una griglia di punti, ognuno descritto da numeri



pixel = «picture element»

colore ambra

$3 \text{ canali} \times 8 \text{ bit} = 24 \text{ bit}$
 $+ 8 \text{ bit di luminosità (trasparenza)} = 32 \text{ bit}$
per pixel
FCA311 è il colore ambra di queste slide

Quanto pesa un'immagine 4K?

Un ordine di grandezza: dall'immagine al flusso di dati verso lo schermo

pixel	$\approx 4000 \times 2000$	8×10^6 pixel
byte per pixel	32 bit / 8	4 byte
una immagine	$8 \times 10^6 \times 4$ byte	≈ 32 MB
100 immagini/s	frequenza di refresh 100 Hz	≈ 3 GB/s

Per questo servono una GPU dedicata e un bus molto veloce tra CPU, memoria e schermo.

La memoria e i suoi indirizzi

Ogni byte della memoria ha un indirizzo, e gli indirizzi crescono di 1 per byte

0x20	0x21	0x22	0x23	0x24	0x25
10110010	01001101	00000000	00000101	11111111	01100001

indirizzo (sopra) → 1 byte = 8 bit (dentro la casella): 0x20, 0x21, 0x22 ... passo 1, non 8

0b110	base 2 = 6	110	base 10 = 110	0x10	base 16 = 16
-------	---------------	-----	------------------	------	-----------------

calcolatore a 32 bit

indirizzi su 32 bit

2^{32} byte = 4 GB di memoria

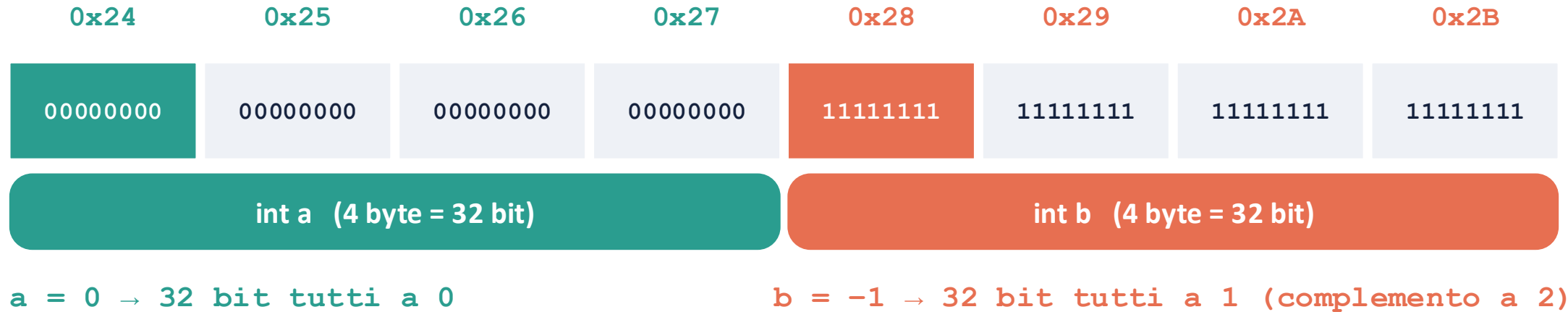
calcolatore a 64 bit

indirizzi su 64 bit

$2^{64} \approx 1.8 \times 10^{19}$ byte = 16 EB

Due interi in memoria

Un int occupa 4 byte, cioè 4 indirizzi consecutivi: il successivo parte 4 indirizzi dopo



Esempio (indirizzi di fantasia)

```
int a = 0;  
int b = -1;
```

```
indirizzo di a: 0x24  
indirizzo di b: 0x28
```

$$0x28 - 0x24 = 4$$

= sizeof(int) byte

L'indirizzo di una variabile è quello del suo primo byte.

Gli stessi numeri come float

0 e -1 con tipo float: i 32 bit cambiano, perché cambia il modo di leggerli

```
float a = 0.0f;
```

0x00000000



```
float b = -1.0f;
```

0xBF800000



$-1.0 = -1.0 \times 2^0$: segno 1, esponente $0 + 127 = 127 = 01111111$, mantissa 0 (l'1 iniziale è implicito)

```
int -1
```

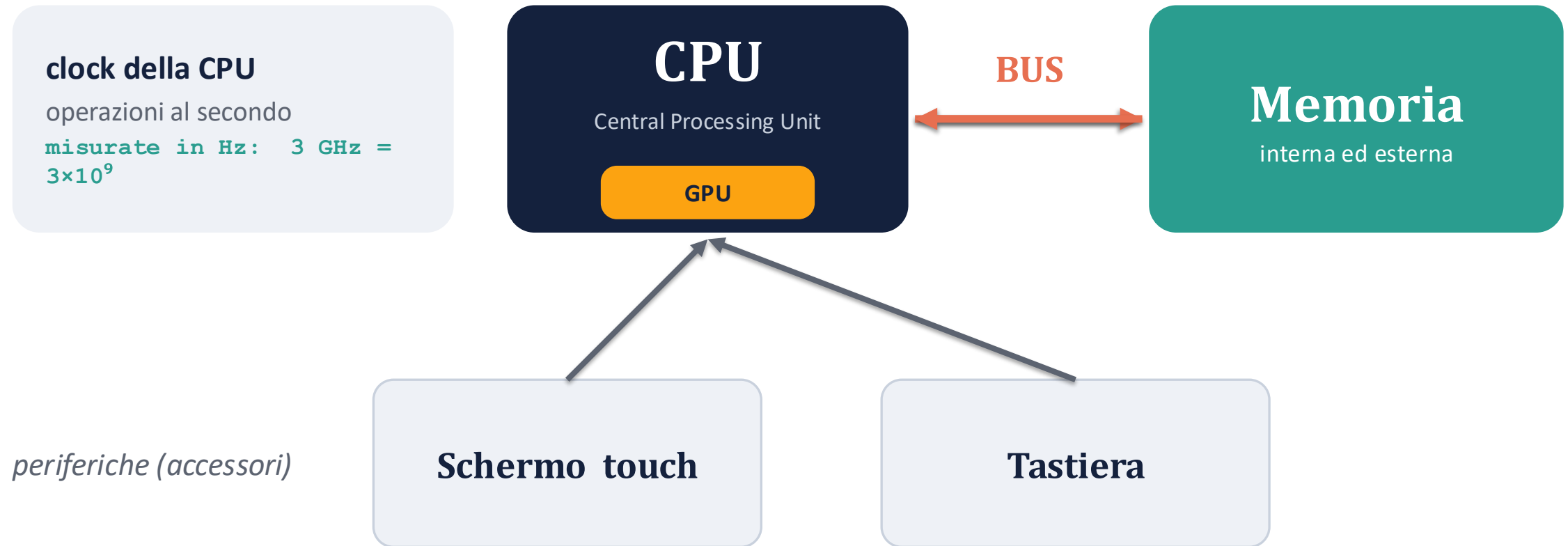
0xFFFFFFFF (complemento a 2)

```
float -1.0
```

0xBF800000 (IEEE 754)

Il calcolatore elettronico

CPU, memoria e periferiche collegate da un bus



GPU = Graphical Processing Unit, dedicata al calcolo delle immagini

Un calcolatore a 8 bit

Due registri, a e b, dentro la CPU; ogni istruzione ha un codice operativo (opCode)

opCode	codice	significato	param.
0000 0000	set_a	scrive in a il valore che segue	1
0000 0001	set_b	scrive in b il valore che segue	1
0000 0010	sum_ab	somma a e b, mette il risultato in a	0
0000 0011	goto	salta alla locazione indicata dal parametro	1
0000 0100	out_a	scrive a sullo schermo	0
0000 0101	inc_a	incrementa di 1 il registro a	0

La CPU legge un opCode e sa cosa fare

Un computer vero ha centinaia di istruzioni

Calcoliamo 3 + 2 in linguaggio macchina

Ogni istruzione modifica i registri; alla fine il risultato va sullo schermo

istruzione	opCode	parametro	registro a	registro b
set_a	0 0 0 0 0 0 0 0	0 0 0 0 0 0 1 1	0 0 0 0 0 0 1 1	—
set_b	0 0 0 0 0 0 0 1	0 0 0 0 0 0 1 0	0 0 0 0 0 0 1 1	0 0 0 0 0 0 1 0
sum_ab	0 0 0 0 0 0 1 0		0 0 0 0 0 1 0 1	0 0 0 0 0 0 1 0
out_a	0 0 0 0 0 1 0 0		0 0 0 0 0 1 0 1	0 0 0 0 0 0 1 0

schermo: 5

0000 0101 in binario è $3 + 2 = 5$: **out_a** lo traduce in cifra decimale e lo mostra.

Il programma in memoria: assembly e assembler

Le istruzioni stanno in memoria come numeri; l'assembly le scrive con nomi leggibili

locazione	contenuto	istruzione
0000 0000	0000 0000	set_a, 3
0000 0001	0000 0011	
0000 0010	0000 0001	set_b, 2
0000 0011	0000 0010	
0000 0100	0000 0010	sum_ab
0000 0101	0000 0100	out_a

registri a e b: dentro la CPU, non in memoria

Assembly

```
set_a, 3
set_b, 2
sum_ab
out_a
```

assembler

linguaggio macchina

Linguaggi di alto livello

Per scrivere programmi veri non bastano set_a, set_b e sum_ab

```
Assembly (4 righe)

set_a, 3
set_b, 2
sum_ab
out_a
```



```
Linguaggio di alto livello

a = 3
b = 2
print(a + b)
```

Vicino alla macchina

poche istruzioni elementari, dipendono dalla CPU

Vicino a noi

una riga = molte istruzioni macchina

C, C++, Python

Il linguaggio C: dichiarare le variabili

In C ogni variabile ha un tipo, e va dichiarata prima di usarla

```
somma.c

int a;           // intero
int b;
double somma;   // reale

a = 3;           // assegnazione
b = 2;
somma = a + b;
printf("%f", somma); // stampa
```

Dichiarazione

tipo + nome: riserva in memoria lo spazio per la variabile

Assegnazione

il simbolo = copia il valore a destra nella variabile a sinistra

Nomi liberi

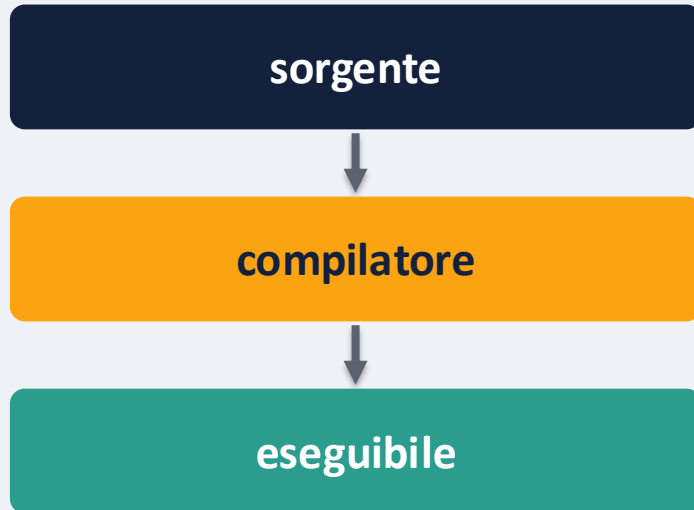
a, somma, caio, pippo: l'importante è il tipo

I tipi (int, double, ...) si vedono nel dettaglio nella prossima lezione.

Compilatore o interprete?

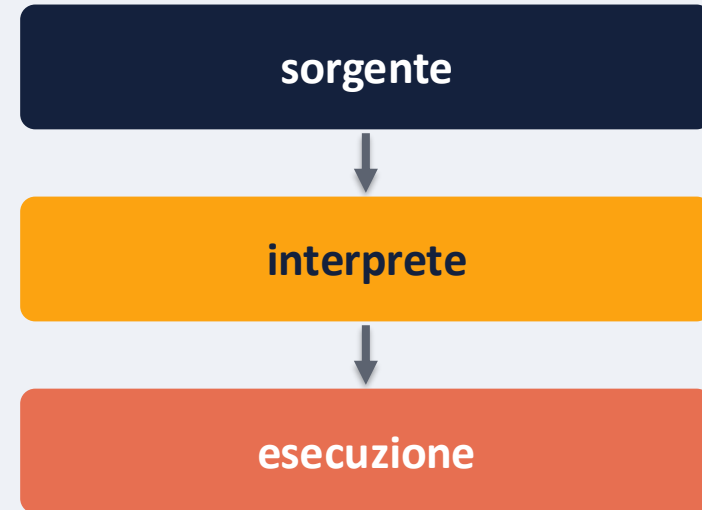
Due modi di trasformare un sorgente in istruzioni per la CPU

Compilatore — C, C++



Traduce tutto una volta sola. Poi si esegue il programma tradotto, tante volte.

Interprete — Python



Traduce ed esegue riga per riga, ogni volta che si lancia il programma.

Dal file .c all'eseguibile

Due passi: compilazione e linking



+ librerie (printf, sqrt ...)

Il compilatore traduce il nostro sorgente in prog.o: manca ancora il codice di printf e delle funzioni matematiche.

Il linker lo unisce alle librerie e produce il file eseguibile.

```
Terminale

$ gcc -c prog.c
$ gcc prog.o -o prog -lm
$ ./prog

# tutto insieme:
$ gcc -Wall -Wextra -o prog prog.c -lm
```

C è tipizzato, Python no

Stessa idea, ma regole diverse sui tipi delle variabili

Python: nessuna dichiarazione

```
a = 2
b = 3
c = a + b      # 5
a = "ciao"
b = 2.1
c = a + b
```

**TypeError: can only concatenate str
(not "float") to str**

l'errore arriva durante l'esecuzione

C: tipo dichiarato

```
int a;
a = 2;
a = "ciao";
```

**warning: incompatible pointer to integer
conversion**

l'errore arriva già in compilazione

Esercizi

Da fare a casa, con carta e penna; poi si può controllare con un programma in C

1 Approssimazione

Scrivi 0.1 in base 2: perché la parte frazionaria non si ferma mai? Spiega $0.1 + 0.2 \neq 0.3$.

2 ASCII

Scrivi in binario a 8 bit 'a' (97) e 'A' (65). In quale bit differiscono? Cosa vale 'a' - 'A'?

3 Immagini

Quanti byte occupa una foto di 12 Mpixel con 24 bit per pixel? E un secondo di video a 30 fotogrammi/s?

4 Macchina a 8 bit

Scrivi in assembly e in binario il programma che calcola $5 + 1$ con `inc_a` e lo mostra sullo schermo.

Riepilogo

1

Reali

float 32 bit, double 64 bit: $(-1)^s \times m \times 2^e$; valori speciali 0, $\pm\infty$, NaN

2

Approssimazione

con cifre finite i risultati sono approssimati: mai confrontare reali con `==`

3

Caratteri e immagini

sono numeri: ASCII e Unicode per il testo, RGB per i pixel

4

Calcolatore

CPU, memoria e bus: istruzioni e dati sono numeri binari con un indirizzo

5

Dal C all'eseguibile

il compilatore traduce, il linker aggiunge le librerie: gcc fa entrambi